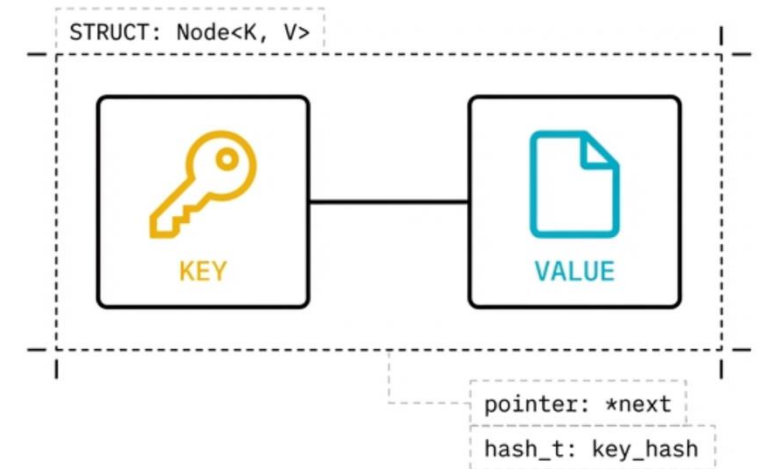
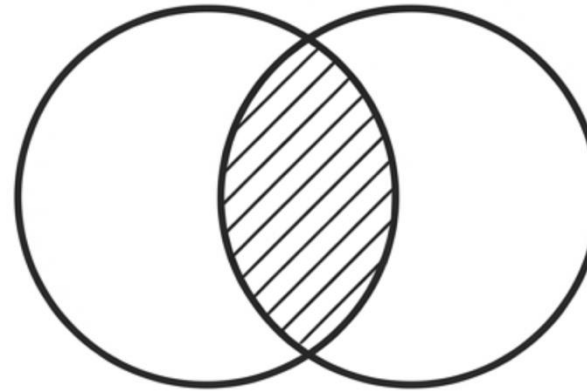
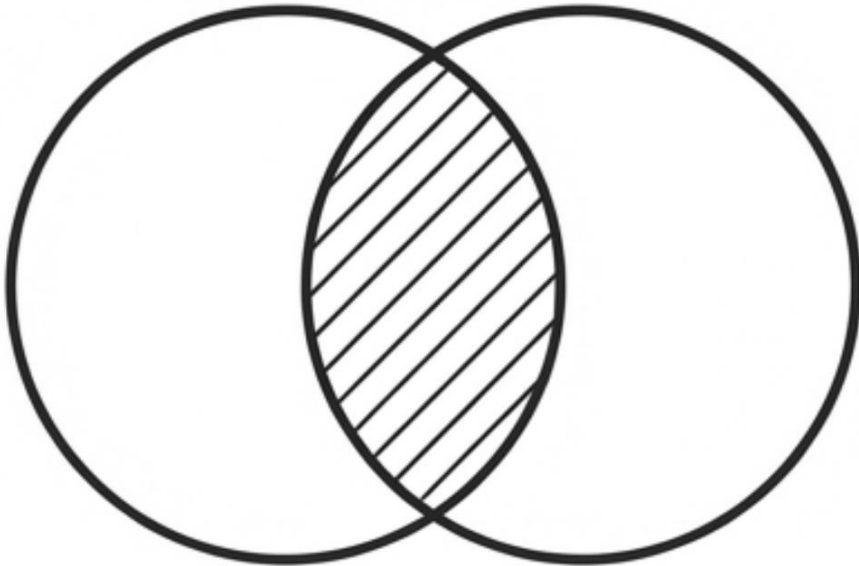


# Conjuntos y Mapas

Aspectos teóricos

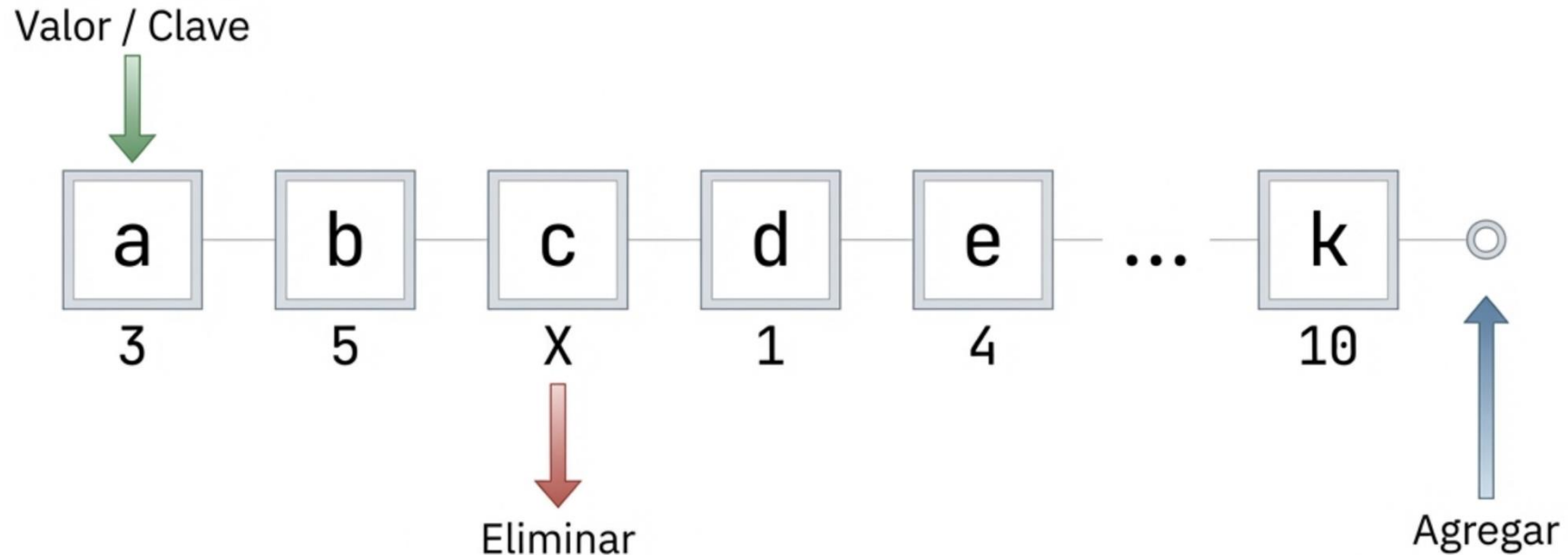


# Conjunto



En ciencias de la computación, la finalidad principal de los conjuntos (sets) es representar y gestionar una colección de elementos únicos, donde cada dato puede aparecer una sola vez.

# Conjunto



A diferencia de las estructuras lineales, las operaciones se centran exclusivamente en el valor de los datos, no en su índice.



# Conjuntos

| Característica    | Arreglos y Listas              | Conjuntos (Sets)                                                                                                                                                                                                 |
|-------------------|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Duplicación       | Permitida y significativa.     |  <b>Estrictamente Prohibida (Únicos).</b>  |
| Ordenamiento      | Importante (Acceso por índice) | <b>Irrelevante (Enfoque en membresía).</b>                                                                                                                                                                       |
| Caso de Uso Ideal | Colecciones secuenciales       | <b>Verificación de unicidad y pertenencia.</b>                                                                                                                                                                   |

# Conjunto

## Plantillas (Templates):

Permite que el conjunto maneje cualquier tipo de dato genérico T.

```
template <typename T>
class ISet {
public:
    ISet() : size(0) {}
    virtual size_t getSize() const { return size; }
    virtual bool isEmpty() const { return size == 0; }
    virtual void clear() = 0;
    virtual void print() const = 0;
    virtual bool add(const T) = 0;
    virtual bool remove(const T) = 0;
    virtual bool contains(const T) const = 0;
protected:
    size_t size;
};
```

## Interfaz Abstracta:

Abstrae la funcionalidad matemática en un contrato C++ estandarizado.

# Conjunto

```
ISet() : size(0) {}  
virtual size_t getSize() const { return size; }  
virtual bool isEmpty() const { return size == 0; }  
// ...  
protected:  
    size_t size;
```

## Estado Protegido

La variable `size` rastrea eficientemente la cantidad de elementos. Al ser `protected`, las clases derivadas pueden acceder a ella directamente.

## Consultas Concretas

A diferencia de las funciones de manipulación de datos, `getSize` e `isEmpty` están implementadas directamente en la interfaz base para evitar duplicación de código en clases derivadas.

# Conjunto

```
virtual void clear() = 0;  
virtual void print() const = 0;  
virtual bool add(const T) = 0;  
virtual bool remove(const T) = 0;  
virtual bool contains(const T) const = 0;
```

## Funciones Virtuales Puras (= 0):

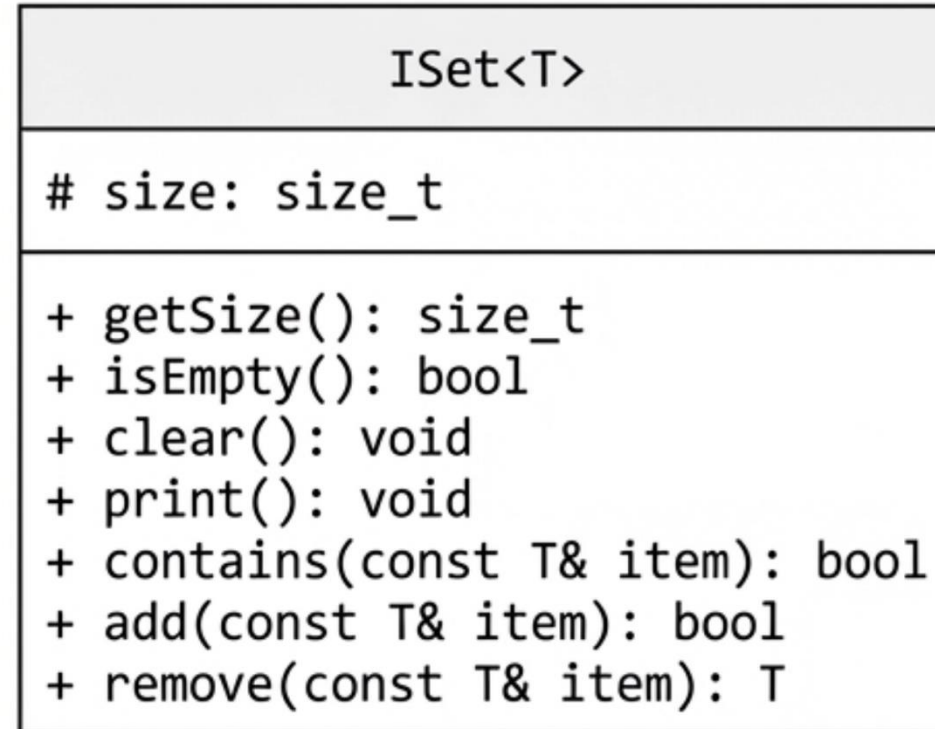
Fuerza a cualquier estructura de datos de conjunto concreta (ej. Hash Table, Árbol) a implementar estas metodologías.

## Retorno Booleano:

add y remove devuelven `bool` para confirmar el éxito de la operación, un aspecto crítico para manejar el principio de unicidad (ej. devuelve `false` si se intenta agregar un duplicado).



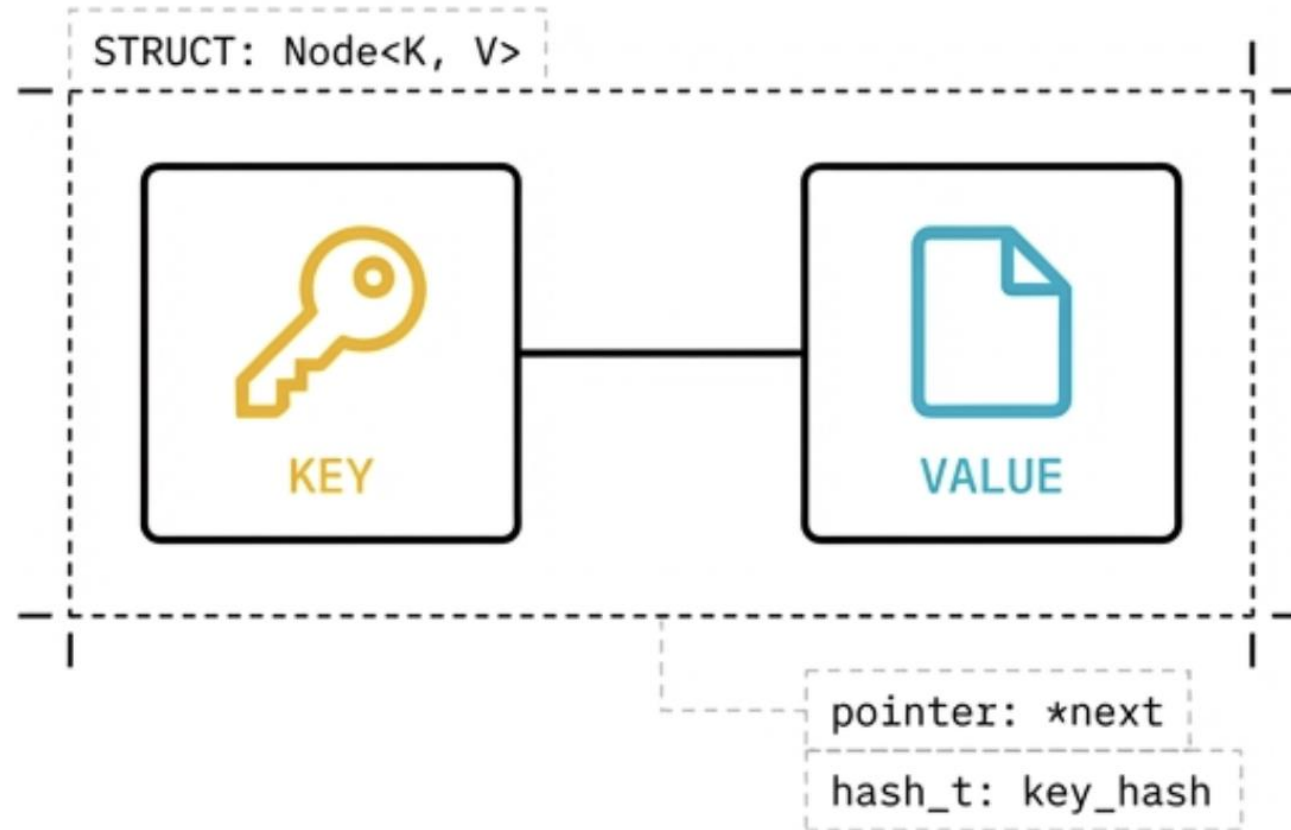
# Conjunto



El diagrama UML encapsula la estructura de la interfaz, sirviendo como el plano definitivo para cualquier clase de implementación.



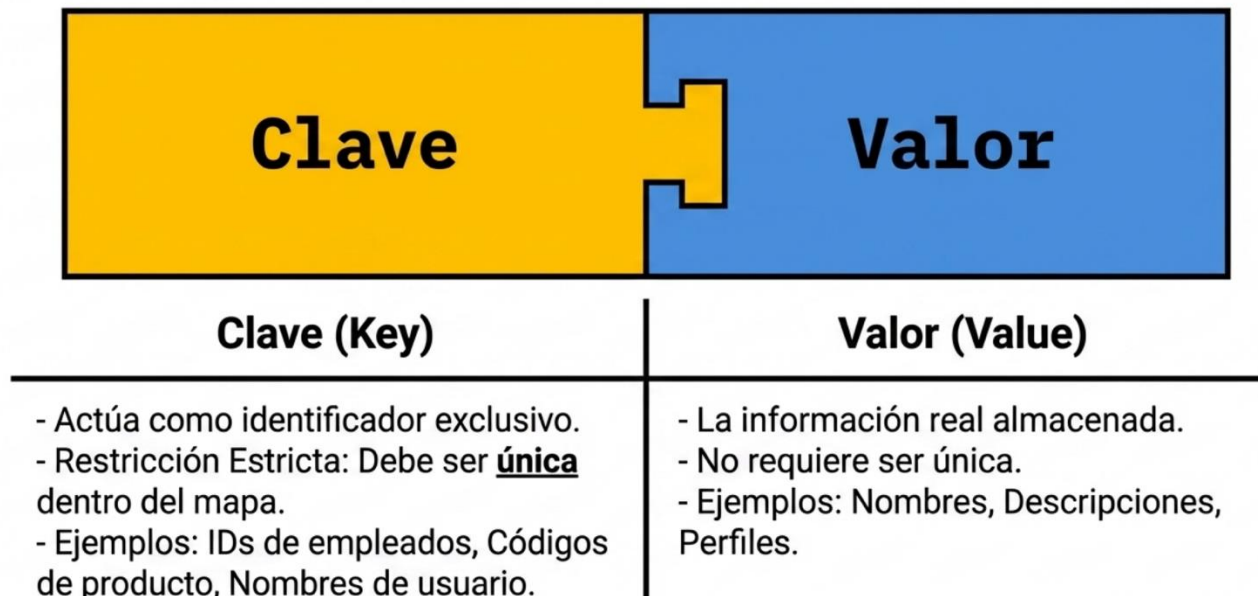
# Mapas



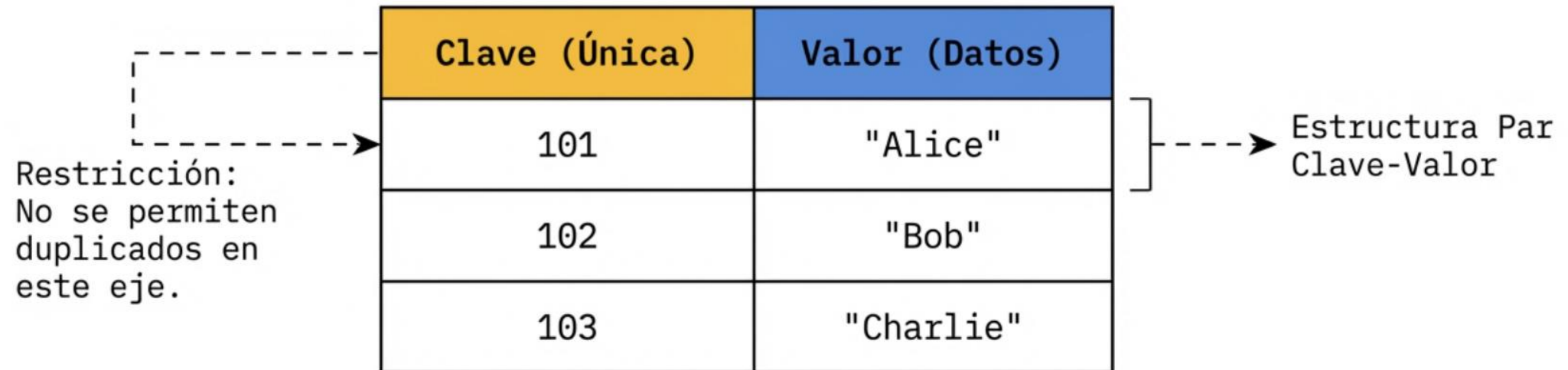
# Mapas

Un Mapa (también conocido como Diccionario o Arreglo Asociativo) es una estructura optimizada para asociar datos con identificadores únicos.

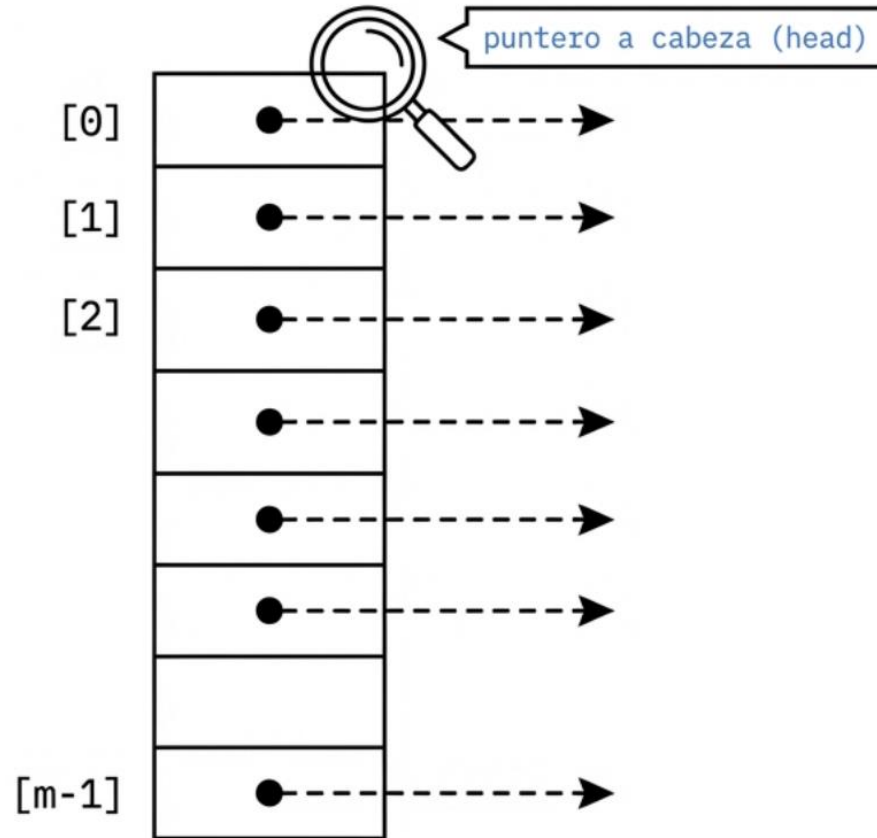
- Provee almacenamiento y recuperación ultra-rápida.
- Casos de uso críticos: Bases de datos, sistemas de caché.



# Mapas



# Mapas



## El Arreglo Base:

El mapa utiliza un arreglo de tamaño fijo ( $m$ ), llamado "cubetas".

## Punteros a Listas:

Cada cubeta actúa como un puntero hacia la cabeza (head) de una lista enlazada.

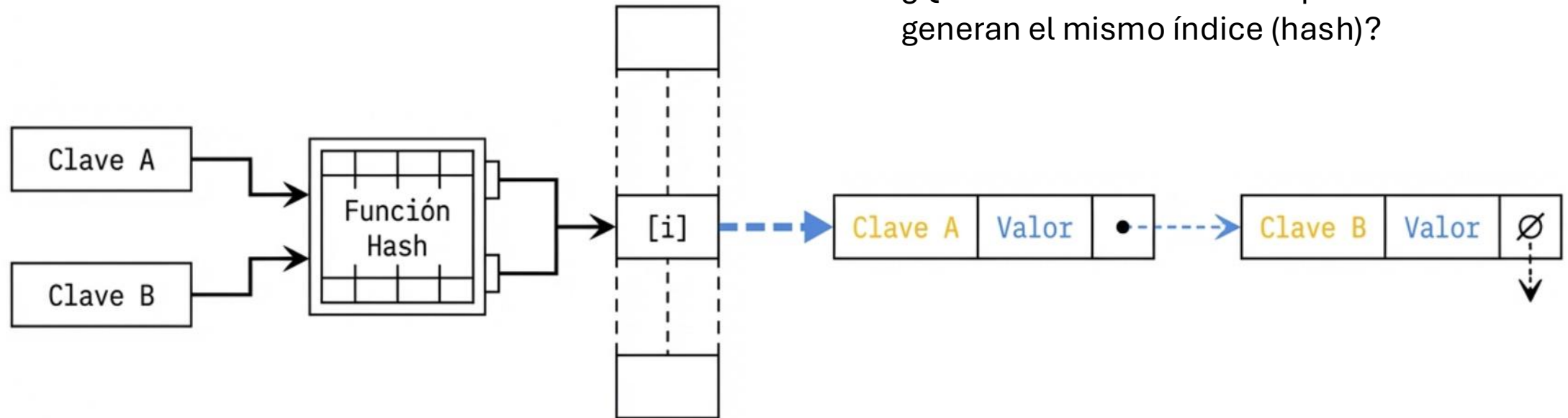
## Rehashing:

Rehashing: Si el número de elementos crece significativamente, el arreglo debe redimensionarse y reorganizar sus claves.

`size = m (Número total de cubetas)`

# Mapas

¿Qué sucede cuando múltiples claves generan el mismo índice (hash)?



1 Múltiples claves son asignadas a la misma cubeta.

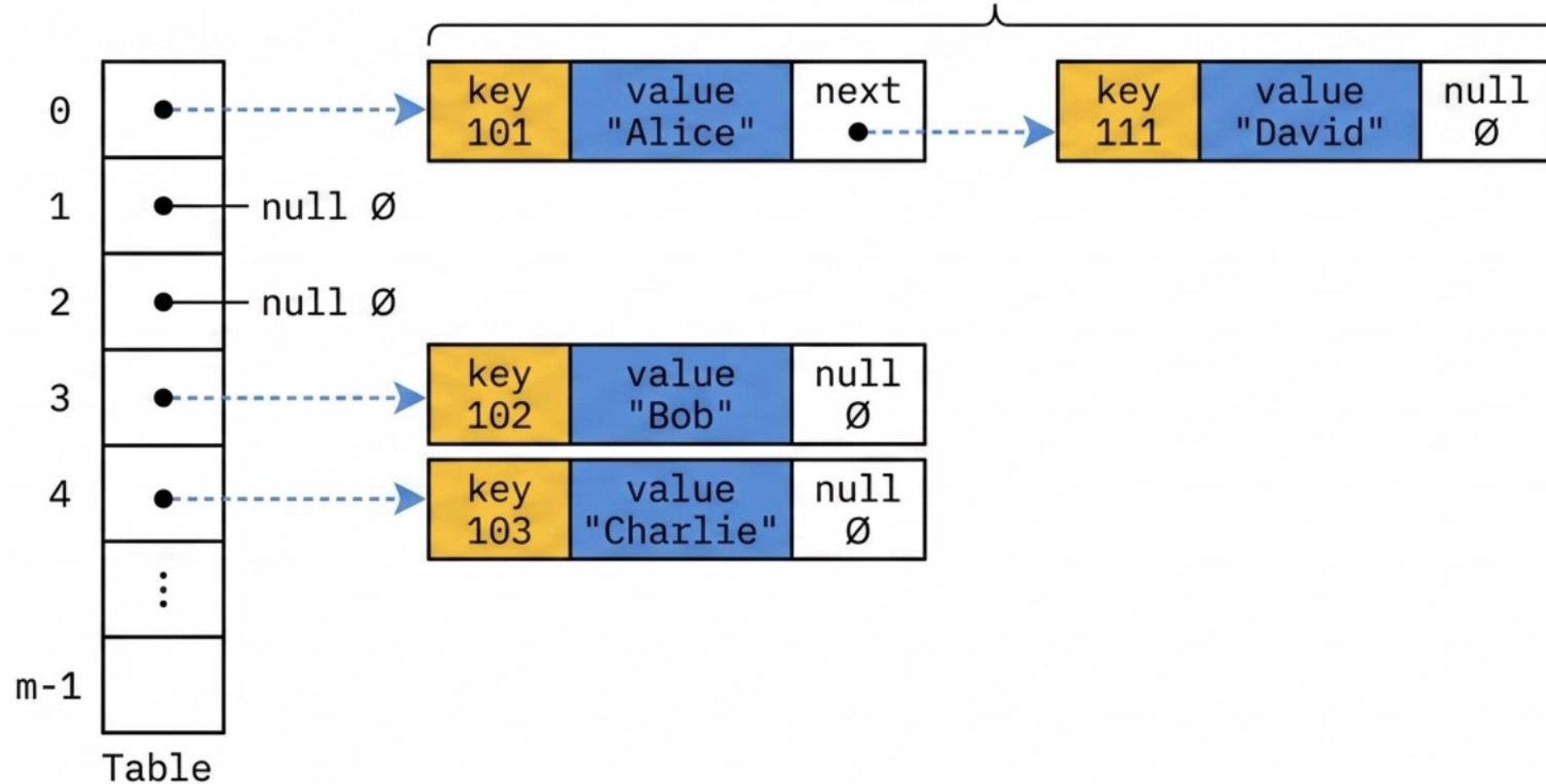
2 En lugar de sobrescribir, el sistema utiliza Encadenamiento Separado (*Separate Chaining*).

3 Los pares Clave-Valor se almacenan como nodos secuenciales en una lista enlazada anclada a esa cubeta.

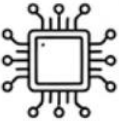
# Mapas

## Cadena (Chain List)

Colisión resuelta: 101 y 111 coexisten en la Cubeta 0 mediante la lista enlazada.  
Separate chaining physical architecture.



# Mapas

| <br>Concepto Lógico | <br>Implementación Física | <br>Restricción del Sistema |
|------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| Arreglo Asociativo                                                                                   | Arreglo de Cubetas (m)<br>+ Listas Enlazadas                                                                 | Requiere rehashing si<br>cambia la capacidad.                                                                  |
| Clave ( <b>Key</b> )                                                                                 | Identificador de<br>Búsqueda                                                                                 | <b>Debe ser absolutamente<br/>única.</b>                                                                       |
| Valor ( <b>Value</b> )                                                                               | Carga Útil (Payload)                                                                                         | Múltiples claves pueden<br>tener el mismo valor.                                                               |